

Example Exam with Solutions

Analytical Skills for Business

© Benjamin Gross

2025-11-06

Table of contents

1	Version Control Systems: Git and GitHub	3
1.1	Question	3
1.2	Solution	3
1.2.1	Part a: Git vs. GitHub	3
1.2.2	Part b: Core Git Concepts	4
1.2.3	Part c: Merging vs. Rebasing	4
2	Programming Languages and Development Environments	5
2.1	Question	5
2.2	Solution	5
2.2.1	Part a: Programming Languages Comparison	5
2.2.2	Part b: Programming Language Elements	6
3	Development environments and tools	7
3.1	Question	7
3.2	Solution	7
3.2.1	Part a: Containers and Docker	7
3.2.2	Part b: IDEs	7
4	No-Code and Low-Code Tools	9
4.1	Question	9
4.2	Solution	9
4.2.1	Part a: No-Code/Low-Code Tools	9
4.2.2	Part b: Advantages of No-Code/Low-Code Tools	10
5	Descriptive Statistics	11
5.1	Question	11

5.2	Solution	11
5.2.1	Part a: Central Tendency Measures	11
5.2.2	Part b: Data Analysis Types	12
6	Data Quality and Association	13
6.1	Question	13
6.2	Solution	13
6.2.1	Part a: Data Quality Issues	13
6.2.2	Part b: Correlation vs. Regression	14
7	R Programming Practical Questions	16
7.1	Part a: Basic Operations	16
7.1.1	Question	16
7.1.2	Solution	16
7.2	Part b: Working Directory	16
7.2.1	Question	16
7.2.2	Solution	16
7.3	Part c: Package Management	16
7.3.1	Question	17
7.3.2	Solution	17
7.4	Part d: Getting Help	17
7.4.1	Question	17
7.4.2	Solution	17
7.5	Part e: Data Frame Creation	17
7.5.1	Question	17
7.5.2	Solution	18
8	References	18

Exam Information

This document contains example exam questions with detailed solutions for the Analytical Skills for Business course at Hochschule Fresenius.

Total Points: 90

Duration: 90 minutes (1 point per minute) **Questions:** 7 sections

1 Version Control Systems: Git and GitHub

Points: 15

1.1 Question

- a) (5 points) Explain the key differences between Git and GitHub. What is the primary purpose of each?
- b) (5 points) Describe three core concepts in Git (repository, commit, branch) and explain their importance in collaborative analytics projects.
- c) (5 points) Explain the difference between merging and rebasing in Git. When would you use one approach over the other?

1.2 Solution

1.2.1 Part a: Git vs. GitHub

Git is a distributed version control system that tracks changes in files and coordinates work among multiple contributors. It:

- Operates locally on your computer
- Maintains a complete project history
- Enables offline work
- Provides branching and merging capabilities

GitHub is a web-based hosting service for Git repositories that adds collaboration features:

- Provides remote repository storage
- Offers pull request workflows for code review
- Includes issue tracking and project management
- Supports automated workflows (GitHub Actions)
- Facilitates documentation through wikis and README files

Key distinction: Git is the version control tool itself, while GitHub is a platform that hosts Git repositories and adds collaborative features.

1.2.2 Part b: Core Git Concepts

1. **Repository:** A central storage location containing all project files and their complete revision history. Repositories enable tracking of all changes, ensuring reproducibility and accountability in data analysis workflows.
2. **Commit:** A snapshot of the project at a specific point in time, representing a set of changes. Commits create a traceable history of modifications, allowing teams to understand when and why changes were made, and to revert to previous states if needed.
3. **Branch:** An independent line of development allowing parallel work on different features. Branches enable teams to work simultaneously on different aspects of projects without interfering with each other's work or the main codebase.

1.2.3 Part c: Merging vs. Rebasing

Merging creates a new commit that combines the histories of two branches:

- Preserves complete history including all branch points
- Creates merge commits
- Shows the actual development timeline
- Safer for shared/public branches

Rebasing rewrites the commit history to create a linear sequence:

- Creates a cleaner, linear project history
- Avoids merge commits
- Makes history easier to follow
- Should be avoided on shared branches as it rewrites history

When to use: Use merging for integrating changes in shared branches and when preserving the complete development history is important. Use rebasing for cleaning up local commits before pushing or when a linear history is preferred for feature branches.

2 Programming Languages and Development Environments

Points: 12

2.1 Question

- a) (6 points) Compare R, Python, and SQL in terms of their primary use cases in business analytics. Provide one specific example for each language.
- b) (6 points) Explain three elements of programming languages (from: syntax, libraries, variables, functions, objects, conditions, loops) and why they are important for data analysis.

2.2 Solution

2.2.1 Part a: Programming Languages Comparison

R: A programming language specifically designed for statistical computing and graphics. Widely used among statisticians and data miners for developing statistical software and data analysis.

- **Primary use:** Statistical analysis, data visualization, academic research
- **Example:** Creating complex visualizations with ggplot2, performing regression analysis, conducting hypothesis tests

Python: A versatile general-purpose programming language with a rich ecosystem for data science. Uses an object-oriented programming paradigm.

- **Primary use:** Data manipulation, machine learning, software development, automation
- **Example:** Building machine learning models with scikit-learn, data processing with Pandas, creating web applications for data products

SQL: The standard language for managing and querying relational databases. Essential for data extraction, transformation, and loading (ETL) processes.

- **Primary use:** Database queries, data retrieval, data manipulation in relational databases
- **Example:** Extracting customer data from PostgreSQL databases, joining multiple tables for analysis, aggregating sales data across time periods

2.2.2 Part b: Programming Language Elements

1. **Variables:** Named storage locations in a program that hold values. Variables are essential for storing intermediate results, parameters, and data during analysis, allowing code to be flexible and reusable across different datasets.
2. **Functions:** Reusable blocks of code that perform specific tasks. Functions promote code modularity, reduce repetition (DRY principle), and make complex analyses more manageable by breaking them into smaller, testable components.
3. **Loops:** Constructs that repeat a block of code multiple times. Loops are crucial for iterating over datasets, performing batch operations, and automating repetitive tasks like processing multiple files or running simulations.

Alternative valid answers:

- **Libraries:** Collections of pre-written code providing specialized functionality, saving time and ensuring reliable implementations of complex algorithms
- **Conditions:** Control flow statements that execute code based on criteria, enabling decision-making logic and data filtering
- **Objects:** Instances of classes encapsulating data and behavior, enabling object-oriented programming approaches

3 Development environments and tools

Points: 15

3.1 Question

- a) (7 points) Define what containers like Docker do in the context of software development and data analytics. Explain their benefits.
- b) (8 points) Explain what an IDE is and name two popular IDEs used for data analytics. Describe one key feature of each.

3.2 Solution

3.2.1 Part a: Containers and Docker

Containers, such as those managed by Docker, are lightweight, portable, and self-sufficient environments that package an application along with its dependencies, libraries, and configuration files. This ensures that the application runs consistently across different computing environments.

Benefits of Containers:

- **Portability:** Containers can run on any system that supports the container runtime, eliminating the “it works on my machine” problem.
- **Isolation:** Containers isolate applications from each other and the host system, improving security and stability.
- **Efficiency:** Containers share the host system’s kernel and resources, making them more lightweight and faster to start compared to virtual machines.
- **Scalability:** Containers can be easily scaled up or down to handle varying workloads, facilitating efficient resource utilization.
- **Reproducibility:** Containers ensure consistent environments for development, testing, and production, reducing bugs related to environment differences.

3.2.2 Part b: IDEs

Integrated Development Environment (IDE): A software application that provides comprehensive facilities to computer programmers for software development. IDEs typically include a code editor, debugger, build automation tools, and a graphical user interface (GUI).

Popular IDEs for Data Analytics:

- **Jupyter Notebook:** An open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text. Key feature: Interactive data exploration and visualization.
- **RStudio:** An IDE specifically designed for R programming, providing tools to help with plotting, history, debugging, and workspace management. Key feature: Integrated support for R Markdown for dynamic report generation.
- **Visual Studio Code** (alternative valid answer): A lightweight but powerful source code editor that supports multiple programming languages through extensions. Key feature: Extensive extension marketplace for language support and tools.

4 No-Code and Low-Code Tools

Points: 10

4.1 Question

- a) (5 points) Name and briefly describe three no-code or low-code tools for data analytics covered in the course (from: n8n, Snowflake, QGIS, Tableau, Power BI, KNIME).
- b) (5 points) Explain the advantages of using no-code/low-code tools in business analytics contexts and when they might be preferred over traditional programming approaches.

4.2 Solution

4.2.1 Part a: No-Code/Low-Code Tools

KNIME: A visual workflow tool for data analytics that allows users to create data processing pipelines through drag-and-drop nodes.

- Use case: Data cleaning, transformation, and machine learning without coding
- Example: Building ETL workflows, handling missing values, performing statistical analysis

Tableau: A data visualization platform enabling interactive dashboard creation and visual analytics.

- Use case: Creating interactive business intelligence dashboards
- Example: Building sales performance dashboards, visualizing KPIs, sharing insights with stakeholders

Power BI: Microsoft's business intelligence platform integrated with the Microsoft ecosystem.

- Use case: Enterprise reporting and data visualization
- Example: Creating interactive reports from multiple data sources, analyzing business metrics

Alternative valid answers:

- **n8n:** Workflow automation tool for connecting services and creating data pipelines
- **Snowflake/Streamlit:** Cloud data platform with integrated app development capabilities
- **QGIS:** Geographic information system for spatial data analysis and mapping

4.2.2 Part b: Advantages of No-Code/Low-Code Tools

Key Advantages:

1. **Accessibility:** Enable non-programmers and business analysts to perform sophisticated data analysis without deep coding skills, democratizing data analytics across organizations.
2. **Speed of Development:** Visual interfaces and pre-built components accelerate development time, allowing rapid prototyping and faster time-to-insight compared to coding from scratch.
3. **Reduced Errors:** Visual workflows and automated processes reduce syntax errors and common programming mistakes, making analytics more reliable.
4. **Collaboration:** Intuitive interfaces facilitate collaboration between technical and non-technical team members, improving communication and shared understanding.
5. **Maintenance:** Visual representations make workflows easier to understand and maintain, reducing documentation burden and onboarding time.

When to Prefer No-Code/Low-Code:

- Quick prototyping and exploratory analysis
- Business users need self-service analytics
- Standard workflows with established patterns
- Time-to-insight is critical
- Team has limited programming expertise
- Integration with enterprise tools is needed

When Traditional Programming is Better:

- Complex custom algorithms required
- Performance optimization critical
- Version control and reproducibility essential
- Advanced statistical or ML techniques needed

5 Descriptive Statistics

Points: 18

5.1 Question

- a) (9 points) Explain three measures of central tendency (mean, median, mode) and when each is most appropriate to use.
- b) (9 points) Distinguish between univariate, bivariate, and multivariate data analysis. Provide one business example for each.

5.2 Solution

5.2.1 Part a: Central Tendency Measures

Mean (Arithmetic Average):

- **Calculation:** Sum of all values divided by the number of observations
- **When to use:** Symmetric distributions without extreme outliers
- **Advantages:** Uses all data points, mathematically convenient
- **Disadvantages:** Sensitive to extreme values
- **Example:** Calculating average sales revenue when distribution is normal

Median (Middle Value):

- **Calculation:** Middle value when data is sorted (50th percentile)
- **When to use:** Skewed distributions or presence of outliers
- **Advantages:** Robust to outliers, better represents “typical” value in skewed data
- **Disadvantages:** Doesn’t use all data points
- **Example:** Measuring typical household income (where billionaires skew the mean)

Mode (Most Frequent Value):

- **Calculation:** Value that appears most frequently
- **When to use:** Categorical data, identifying most common category
- **Advantages:** Works with any data type, identifies popular values
- **Disadvantages:** May not exist or may not be unique
- **Example:** Finding most popular product category or preferred customer segment

5.2.2 Part b: Data Analysis Types

Univariate Analysis:

- **Definition:** Examines one variable at a time to understand distribution, central tendency, and variability
- **Techniques:** Summary statistics (mean, median, SD), histograms, box plots, bar charts
- **Business example:** Analyzing monthly sales revenue to identify typical performance, seasonality, and outliers. Answers “What is typical revenue?” and “How much does it vary?”

Bivariate Analysis:

- **Definition:** Examines relationships between two variables to identify associations, correlations, or dependencies
- **Techniques:** Correlation coefficients, scatter plots, cross-tabulation, simple linear regression
- **Business example:** Examining relationship between advertising spend and sales revenue to determine campaign effectiveness. Answers “How do these variables relate?” and “Does one predict the other?”

Multivariate Analysis:

- **Definition:** Examines three or more variables simultaneously to understand complex relationships and patterns
- **Techniques:** Multiple regression, PCA, cluster analysis, correlation matrices, heatmaps
- **Business example:** Analyzing customer demographics, purchase history, website behavior, and satisfaction scores simultaneously to identify market segments and predict customer lifetime value. Answers “How do multiple factors jointly influence outcomes?”

6 Data Quality and Association

Points: 12

6.1 Question

- a) (6 points) Describe three common data quality issues (from: missing values, inconsistent data, duplicate records, outliers) and appropriate strategies for handling each.
- b) (6 points) Explain the difference between correlation and regression analysis. Why is it important to remember that “correlation does not imply causation”?

6.2 Solution

6.2.1 Part a: Data Quality Issues

Missing Values:

- **Problem:** Complete absence of data, represented as NULL, NA, empty strings, or placeholder values
- **Patterns:** MCAR (Missing Completely At Random), MAR (Missing At Random), MNAR (Missing Not At Random)
- **Handling strategies:**
 - Deletion (listwise/pairwise) when data is MCAR and missing percentage is low
 - Imputation: mean/median/mode for numerical data
 - Forward/backward fill for time series
 - Model-based imputation (regression, KNN, multiple imputation)
 - Flag creation to preserve information about missingness

Inconsistent Data:

- **Problem:** Format variations, unit inconsistencies, date format variations, encoding issues
- **Examples:** “New York” vs. “NY” vs. “new york”; mixing metric and imperial units
- **Handling strategies:**
 - Standardization to consistent formats and units
 - Text normalization (lowercasing, trimming whitespace)
 - Date parsing with explicit format specifications
 - Encoding conversion and validation

Duplicate Records:

- **Problem:** Exact duplicates, near duplicates with variations, or same entity with different identifiers
- **Handling strategies:**
 - Exact match removal using unique identifiers
 - Fuzzy matching for near duplicates (Levenshtein distance, Jaccard similarity)
 - Record linkage and deduplication algorithms
 - Master data management (MDM) approaches

Outliers and Anomalies (alternative valid answer):

- **Problem:** Statistical outliers beyond expected range, domain outliers impossible in context, data entry errors
- **Handling strategies:**
 - Detection using Z-score, IQR method, isolation forests, DBSCAN
 - Investigation to determine if errors or valid extreme values
 - Treatment: removal, capping/winsorizing, transformation, or separate analysis
 - Domain validation with business rules

6.2.2 Part b: Correlation vs. Regression

Correlation Analysis:

- Measures strength and direction of linear relationships between two variables
- Produces coefficient ranging from -1 (perfect negative) to +1 (perfect positive)
- Symmetric relationship (X-Y same as Y-X)
- No distinction between dependent and independent variables
- Describes strength and direction only
- Example: Pearson correlation between advertising spend and sales = 0.82 suggests strong positive relationship

Regression Analysis:

- Models relationships between variables to predict outcomes
- Asymmetric relationship (predicting Y from X)
- Clear dependent (outcome) and independent (predictor) variables
- Provides prediction equation: $Y = \text{intercept} + X \times \text{slope}$
- Estimates effect sizes, intercepts, and enables statistical inference
- Example: $\text{Sales} = 1000 + 5.2 \times \text{Advertising_Spend}$ predicts sales increase of 5.2 units per unit increase in advertising

“Correlation Does Not Imply Causation”:

Why This Is Critical:

1. **Confounding variables:** Third variables may cause both X and Y to vary together
 - Example: Ice cream sales and drowning rates both increase in summer (temperature is the confounder)
2. **Reverse causation:** Y might cause X instead of X causing Y
 - Example: Correlation between employee satisfaction and productivity—does satisfaction drive productivity or vice versa?
3. **Spurious correlations:** Random chance or coincidence can create correlations
 - Example: Number of Nicolas Cage films correlates with swimming pool drownings
4. **Common response:** Both variables respond to the same underlying factor
 - Example: Umbrella sales and flood insurance claims both respond to rainfall

Business Implications:

- Strong correlation suggests investigation is warranted but not action
- Establishing causation requires: temporal precedence, experimental design (A/B testing), controlling confounders, theoretical justification
- Making causal claims from correlation alone can lead to ineffective or harmful business decisions

Example: A company observes high correlation ($r = 0.85$) between office size and employee productivity. Incorrectly assuming causation, they invest millions in larger offices, only to find no productivity gains. The true driver was company growth—successful growing companies both hire more productive employees AND need larger offices.

7 R Programming Practical Questions

Points: 8

7.1 Part a: Basic Operations

Points: 2

7.1.1 Question

Write R code to calculate and store the following result in a variable named `result`:

$$3 + \sqrt{9} - 2^2 \cdot \frac{4}{3}$$

7.1.2 Solution

```
result <- 3 + sqrt(9) - 2^2 * 4/3
```

7.2 Part b: Working Directory

Points: 1

7.2.1 Question

Write the R code to get the path of your current working directory.

7.2.2 Solution

```
getwd()
```

7.3 Part c: Package Management

Points: 1

7.3.1 Question

Write the R code to install the `tidyverse` package.

7.3.2 Solution

```
install.packages("tidyverse")
```

7.4 Part d: Getting Help

Points: 1

7.4.1 Question

Write the R code to access the help documentation for the `mean()` function.

7.4.2 Solution

```
?mean  
# or  
help(mean)
```

7.5 Part e: Data Frame Creation

Points: 3

7.5.1 Question

Create a data frame in R containing the following sales data:

Region	Sales
North	45000
South	38000
East	52000

Store it in a variable named `sales_data`.

7.5.2 Solution

```
library(tidyverse)

Region <- c("North", "South", "East")
Sales <- c(45000, 38000, 52000)
sales_data <- tibble(Region, Sales)

# Alternative using base R data.frame:
# sales_data <- data.frame(
#   Region = c("North", "South", "East"),
#   Sales = c(45000, 38000, 52000)
# )
```

8 References

Cetinkaya-Rundel, M., & Hardin, J. (2021). *Introduction to modern statistics* (2nd ed.). OpenIntro.

Illowsky, B., & Dean, S. (2018). *Introductory statistics*. OpenStax.

GeeksforGeeks. (2024). *Understanding hypothesis testing*. <https://www.geeksforgeeks.org/software-testing/understanding-hypothesis-testing/>

GitHub Docs. (2024). *About GitHub and Git*. <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>